ScholarPath — From EERD to Relational Schema

Step 3: Relational Mapping

ScholarPath Team

Abstract

This is Part 3 of a four-part teaching series that walks the ScholarPath data model from a plain Entity–Relationship Diagram (ERD) through an Enhanced ER Diagram (EERD), then into a *relational schema*, and finally into object-oriented Class Diagrams. Here we take the EERD produced in Part 2 and convert it into concrete relations (tables, keys, and foreign keys) by applying Elmasri & Navathe's classic seven/nine-step ER/EER-to-relational mapping algorithm. Every rule is illustrated with a real ScholarPath construct, and every claim is reconciled against the *live* Azure SQL schema (the deployed EF Core model snapshot), so the mapping you see is the schema that actually runs in production — 62 tables in total. Where ScholarPath deliberately deviates from the textbook (multivalued attributes stored as JSON, a single-table specialization, loose references on high-volume tables), the deviation is stated plainly rather than hidden. The companion documents are *ScholarPath — ERD* (Part 1), *ScholarPath — EERD* (Part 2), and *ScholarPath — Class Diagrams* (Part 4).

Contents

1 Goal: turning the EERD into relations

The EERD from Part 2 is a *conceptual* model: it speaks of entities, relationships, cardinalities, weak entities, and specialization. A relational database, by contrast, knows only *relations* (tables), *attributes* (columns), *primary keys*, and *foreign keys*. Step 3 is the disciplined translation between the two worlds.

We follow the standard algorithm presented by Elmasri & Navathe (*Fundamentals of Database Systems*), commonly stated in seven core steps (plus two for the EER extensions of specialization and union), giving the "seven/nine-step" mapping. The algorithm is attractive for a defense because it is *mechanical and auditable*: each conceptual construct has exactly one prescribed rule, so a reviewer can trace any table in the schema back to the EERD element that produced it.

Two ground rules shaped our application of the algorithm:

- **Fidelity to the deployed schema.** The result was reconciled against the live EF Core model snapshot (`ApplicationDbContextModelSnapshot.cs`) and the production Azure SQL dump. Where the textbook ideal and the shipped schema differ, we document the *shipped* schema and flag the deviation.
- **Honesty about denormalization.** A graduation project that claims perfect third-normal-form purity invites a hard question at defense. Instead we record the pragmatic choices (JSON multivalued columns, single-table inheritance, loose references) and justify each.

A short notation reminder, used throughout this document and in the figures:

- Underlined attribute = primary key. All primary keys are `Guid` (`uniqueidentifier`) unless noted; junction tables use a *composite* primary key.
- *Italic* attribute = foreign key, with its delete rule (**Cascade**, **Restrict**, or **SetNull**).
- **(soft)** marks a *loose reference*: a `Guid` that points at another relation logically but carries *no* `FOREIGN KEY` constraint — integrity is enforced in application code. This is a deliberate decoupling for high-volume / audit / analytics relations.
- ⋆`UK` = unique index/constraint; `FUK` = *filtered* (partial) unique index; `enc` = encrypted column; `cents` = money stored as integer cents.
- `{audit}` abbreviates the auditing columns (`CreatedAt`, `CreatedByUserId`, `UpdatedAt`, `UpdatedByUserId`, `RowVersion`); `{softdel}` abbreviates the soft-delete columns (`IsDeleted`, `DeletedAt`, `DeletedByUserId`).

2 The mapping algorithm at a glance

Table 1 restates Elmasri & Navathe's nine steps and, for each, names the place(s) in ScholarPath where that construct actually appears. The rest of the document then walks the constructs one by one.

Table 1: The ER/EER-to-relational mapping algorithm, applied to ScholarPath.

Step	Construct	Rule applied	Where it appears in ScholarPath
1	Regular (strong) entity	One relation; choose a primary key.	`Users`, `Scholarships`, `Bookings`, … (every strong entity).
2	Weak entity	Relation including the owner's PK as part of an identifying FK; identity = owner PK + partial key.	`Scholarship_Children`, `Application_Children`, `Education_Entries`, `Resource_Chapters`, `*_Files/_Links`.
3	1:1 relationship	FK (plus ⋆`UK`) on the side with total participation.	`Users`–`UserProfiles`; `Bookings`–`ConsultantReviews`; `Applications`–`CompanyReviews`; `AiInteractions`–`AiRedactionAuditSamples`; `Users`–`UserRiskFlags`.
4	1:N relationship	FK on the N-side referencing the 1-side.	`Scholarships`→`Applications`, `Users`→`Bookings`, `ForumPosts`→`ForumVotes`, …
5	M:N relationship	New relationship relation with composite PK = the two FKs.	`UserRoles` (User ⋈ Role), `ForumPostTags` (Post ⋈ Tag); `SavedScholarships` is the same shape but uniqueness-only (loose).
6	Multivalued attribute	Separate relation keyed by (owner PK + value).	Realized as JSON columns (`PreferredCountriesJson`, `TagsJson`, …) *or* child rows (`Scholarship_Children`) — see the deviation note.
7	N-ary relationship	New relation with a FK to each participant.	`CompanyReviewRequests` (Student × Company × Scholarship × Payment).
8	Specialization / generalization (EER)	*Option C* — single relation with a type discriminator.	User **ISA** {Student, Company, Consultant, Admin}: one `Users` table + role rows in `UserRoles`; role-specific columns folded into one `UserProfiles` relation (NULL when not applicable).
9	Union / category (EER)	—	Not used (no category/union types in the model).

3 Walking each construct with ScholarPath examples

3.1 Step 1 — Strong (regular) entities

Each strong entity becomes exactly one relation, and we pick its primary key. ScholarPath uses surrogate `Guid` keys throughout (rather than natural keys), which keeps foreign keys uniform and avoids leaking business meaning into identifiers. `Users`, `Scholarships`, `Bookings`, `Resources`, `ForumPosts`, `Payments`, and the lookup tables (`Categories`, `Roles`, `ForumTags`, `ExpertiseTags`) are all step-1 relations.

The principal example is the `Users` relation, which carries the ASP.NET Identity attributes:

```
USERS( Id, Email, NormalizedEmail, FirstName, LastName, PasswordHash,
       SecurityStamp, ConcurrencyStamp, EmailConfirmed, PhoneNumber,
       LockoutEnd, LockoutEnabled, AccessFailedCount, TwoFactorEnabled,
       ProfileImageUrl, AccountStatus, ActiveRole, IsOnboardingComplete,
       PreferredLanguage, CountryOfResidence, LastLoginAt, {audit}, {softdel} )
```

A subtle, defense-relevant point: e-mail uniqueness here is *not* a database unique constraint. Identity creates a unique filtered index on `NormalizedUserName` but a *non-unique* index on `NormalizedEmail`; e-mail uniqueness is enforced by the Identity validator in application code. This is the first of several places where "what the model promises" and "what the database enforces" diverge — a theme we return to in Section 6.

3.2 Step 2 — Weak entities and identifying cascades

A weak entity has no key of its own; it is identified only *relative to its owner*. The rule produces a relation that embeds the owner's primary key as part of an identifying foreign key, and — crucially — that foreign key cascades on delete, because a weak-entity row is meaningless once its owner is gone.

ScholarPath's clearest weak entities are the `*_Children` and `Education` rows:

```
EDUCATION_ENTRIES( Id, UserProfileId, InstitutionName, Degree, FieldOfStudy,
       StartDate, EndDate, Gpa(4,2), Description, {audit} )
    FK UserProfileId -> USER_PROFILES(Id)  [Cascade]      -- identifying owner

SCHOLARSHIP_CHILDREN( Id, ScholarshipId, ChildType, KeyEn, KeyAr,
       ValueEn, ValueAr, SortOrder )
    FK ScholarshipId -> SCHOLARSHIPS(Id)  [Cascade]       -- weak EAV entity

APPLICATION_CHILDREN( Id, ApplicationTrackerId, ChildType, Title, Content,
       MetadataJson, OccurredAt, ActorUserId(soft), SortOrder )
    FK ApplicationTrackerId -> APPLICATIONS(Id)  [Cascade] -- status history / notes
```

Note that `Education_Entries` hangs off `UserProfiles`, not directly off `Users` — educational history is owned by the profile. The `Scholarship_Children` and `Application_Children` relations are entity–attribute–value (EAV) carriers: a single weak relation holds many heterogeneous child rows discriminated by a `ChildType` column (e.g. a scholarship's benefits and eligibility bullets, or an application tracker's status-history and note events). The same step-2 pattern produces `Resource_Chapters` (owned by `Resources`) and the `UpgradeRequest_Files`/`UpgradeRequest_Links` relations (owned by `UpgradeRequests`). In every case the identifying FK is **Cascade**, faithfully realizing the existence-dependency of the EERD.

3.3 Step 3 — 1:1 relationships

For a one-to-one relationship the algorithm places the foreign key on the side with *total* participation and backs it with a unique constraint so the "one" is enforced. ScholarPath has five textbook 1:1 relationships:

- `Users`–`UserProfiles`: every profile belongs to exactly one user; the FK `UserId` carries a `*UK`.
- `Bookings`–`ConsultantReviews`: at most one review per completed booking.
- `Applications`–`CompanyReviews`: at most one company review per application.
- `AiInteractions`–`AiRedactionAuditSamples`: at most one privacy-audit sample per AI interaction.
- `Users`–`UserRiskFlags`: at most one churn-risk flag row per user.

```
USER_PROFILES( Id, UserId*UK, Biography enc, ..., {audit} )
    FK UserId -> USERS(Id)  [Restrict]      -- 1:1, *UK enforces "exactly one"

CONSULTANT_REVIEWS( Id, BookingId*UK, StudentId, ConsultantId, Rating, ... )
    FK BookingId -> BOOKINGS(Id)  [Restrict]  -- *UK => one review per booking
```

The unique index on the FK is what upgrades a plain 1:N foreign key into a 1:1 relationship at the database level. (All of these FKs into `Users` use **Restrict**, for reasons explained in Section 5.)

3.4 Step 4 — 1:N relationships

The most common construct: place the foreign key on the "many" side, referencing the "one" side. Examples are everywhere — `Scholarships`→`Applications`, `Users`→`Bookings`, `ForumPosts`→`ForumVotes`, `Resources`→`ResourceChapters`, and the self-reference `ForumPosts`→`ForumPosts` (a reply points to its parent post; a `NULL` parent marks a root thread). Many of these FKs are paired with composite indexes (e.g. `(Status, Deadline)` on `Scholarships`) to support the common query paths.

3.5 Step 5 — M:N relationships (junction relations)

A many-to-many relationship cannot be represented by a single foreign key, so the algorithm introduces a new *relationship relation* (junction table) whose primary key is the composite of the two foreign keys. ScholarPath has two true junction tables:

```
USER_ROLES( UserId, RoleId )                -- M:N junction (User <-> Role)
    FK UserId -> USERS(Id)  [Restrict]      -- NO ACTION: no FK cascades from Users
    FK RoleId -> ROLES(Id)  [Cascade]
    PK = (UserId, RoleId)

FORUM_POST_TAGS( ForumPostId, ForumTagId )   -- M:N junction (Post <-> Tag)
    FK ForumPostId -> FORUM_POSTS(Id)  [Cascade]
    FK ForumTagId  -> FORUM_TAGS(Id)   [Cascade]
    PK = (ForumPostId, ForumTagId)
```

A near-miss worth pointing out is `SavedScholarships` (a student's bookmarked scholarships). It is conceptually an M:N association, and it uses the same shape, *but* it is realized with **uniqueness only**: it has its own surrogate `Id`, a real FK to `Scholarships`, and a unique index on `(UserId, ScholarshipId)` — while the `UserId` leg is a *loose* reference (no FK). This keeps the bookmark table decoupled from the `Users` principal, consistent with the loose-reference strategy of Section 5.

3.6 Step 6 — Multivalued attributes (the JSON deviation)

The textbook rule for a multivalued attribute is to create a *separate* relation keyed by (owner PK + value). ScholarPath *deliberately deviates* from this for most multivalued attributes, storing them instead as JSON string columns: a student's `PreferredCountriesJson` and `PreferredFieldsJson`

on `UserProfiles`, a scholarship's `TagsJson` and `TargetCountriesJson`, a payout's `IncludedPaymentIdsJson`, and so on.

This is a normalization trade-off, and it is stated honestly so the mapping matches the deployed schema. The reasoning:

- These collections are read and written *as a whole* with their owner, are never the subject of a relational join, and rarely need independent querying — so a child relation would add cost without benefit.
- Where a multivalued attribute *did* need querying, ordering, and sharing, we applied the textbook rule properly: community tags became the real `ForumTags` relation plus the `ForumPostTags` M:N junction of Step 5, rather than a `TagsJson` blob.
- Where a multivalued attribute is genuinely heterogeneous and ordered, it became a weak child relation instead (`Scholarship_Children` — Step 2).

In short: JSON for "carried with the owner," a child/junction relation for "queried on its own."

3.7 Step 7 — N-ary relationships

When a relationship genuinely connects more than two participants, the algorithm creates a relation with a foreign key to each participant. ScholarPath's paid company-review workflow is exactly such an n-ary relationship: a single request ties together a **Student**, a **Company**, a **Scholarship**, and a **Payment**.

```
COMPANY_REVIEW_REQUESTS( Id, StudentId, CompanyId, ScholarshipId,
       ApplicationTrackerId(soft), PaymentId, Status,
       ReviewFeeUsdSnapshot(10,2), Currency, SubmittedAt, AcceptedAt,
       RejectedAt, CompletedAt, ClosedAt, CancelledAt, ExpiredAt,
       PendingExpiresAt, CancelReason, RejectReason, {audit}, {softdel} )
    FK StudentId     -> USERS(Id)          [Restrict]
    FK CompanyId     -> USERS(Id)          [Restrict]
    FK ScholarshipId -> SCHOLARSHIPS(Id)   [Restrict]
    FK PaymentId     -> PAYMENTS(Id)       [SetNull]
    (filtered-unique) (StudentId, ScholarshipId)
        WHERE Status IN ('Draft','Submitted','Pending','UnderReview')
```

The four foreign keys are the literal realization of the four participants of the n-ary relationship. The filtered-unique index expresses a business rule "one open review request per (student, scholarship)" and — unlike the analogous rule on `Applications` — it *is* enforced in the database (Section 6).

3.8 Step 8 — Specialization (single-table, Option C)

The EER specialization `User ISA {Student, Company, Consultant, Admin}` has three standard mapping options: (A) one relation per subclass plus the superclass, (B) one relation per subclass only, or (C) a single relation for the whole hierarchy with a type discriminator. ScholarPath chose **Option C**.

Concretely:

- The hierarchy collapses into one `Users` table; the "which subclass(es)" question is answered by role rows in `UserRoles` (and the cached `ActiveRole` discriminator column on `Users`).

- All role-specific attributes are folded into one `UserProfiles` relation, NULL when not applicable to the user's role: student fields (`AcademicLevel`, `Gpa`, `PreferredCountriesJson`), company fields (`OrganizationLegalName`, `OrganizationVerificationStatus`, `CompanyAverageRating`), and consultant fields (`SessionFeeUsd`, `ExpertiseTagsJson`, `StripeConnectAccountId`).

Option C was chosen because a ScholarPath user can hold more than one role over time (a student may upgrade to consultant), which makes rigid one-table-per- subclass mappings awkward; a single table with a role discriminator models the overlap cleanly and keeps authentication on one principal. The cost — NULL columns for inapplicable roles — is acceptable for a profile table that is read far more than it is written.

4 The resulting relational schemas (six areas)

The full schema is large, so it is presented as six area diagrams. Each figure shows the relations, their primary keys, foreign keys (with delete rules), and the unique/filtered indexes for one functional area. Together they constitute the complete mapping of the Part 2 EERD.

4.1 Identity, Access & Profile

This area realizes Step 8 (the single-table user specialization), the 1:1 `Users`–`UserProfiles` relationship (Step 3), the `UserRoles` M:N junction (Step 5), and the `Education_Entries` weak entity (Step 2), alongside the ASP.NET Identity satellite relations and the auth/security tables (refresh tokens, login attempts, password-reset tokens, upgrade requests).

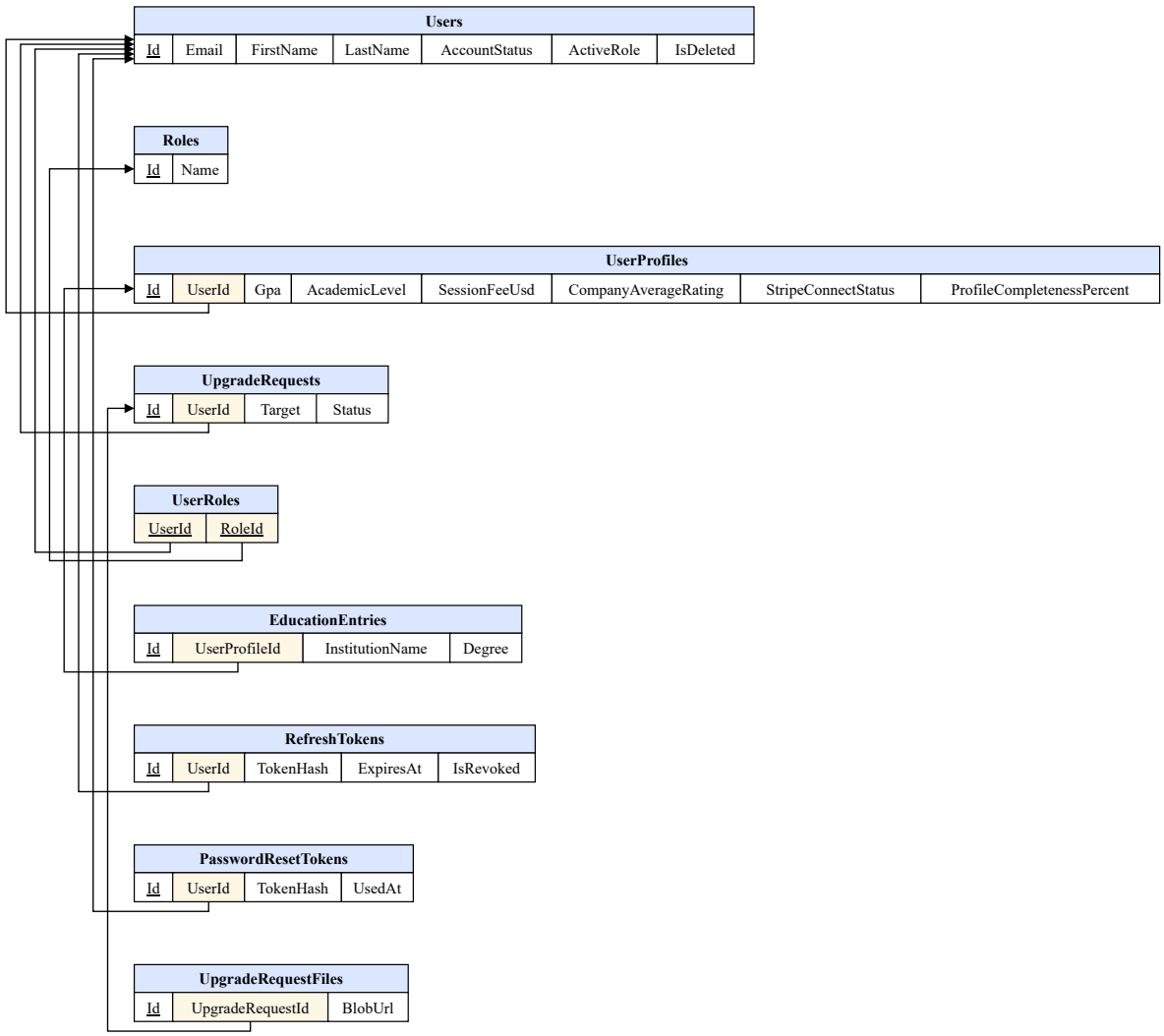

Figure 1: Relational mapping — Identity, Access & Profile. The `Users`/`UserProfiles` pair shows the single-table specialization (Step 8) plus the 1:1 link (Step 3); `UserRoles` is the M:N junction (Step 5); `Education_Entries` is the cascading weak entity (Step 2).

4.2 Scholarships, Applications & Documents

Here `Scholarships`→`Applications` is the central 1:N (Step 4), `Scholarship_Children` and `Application_Children` are weak EAV entities (Step 2), `SavedScholarships` is the uniqueness-only M:N bookmark (Step 5), `CompanyReviewRequests` is the n-ary relationship (Step 7), and `CompanyReviews` is the 1:1 review (Step 3).

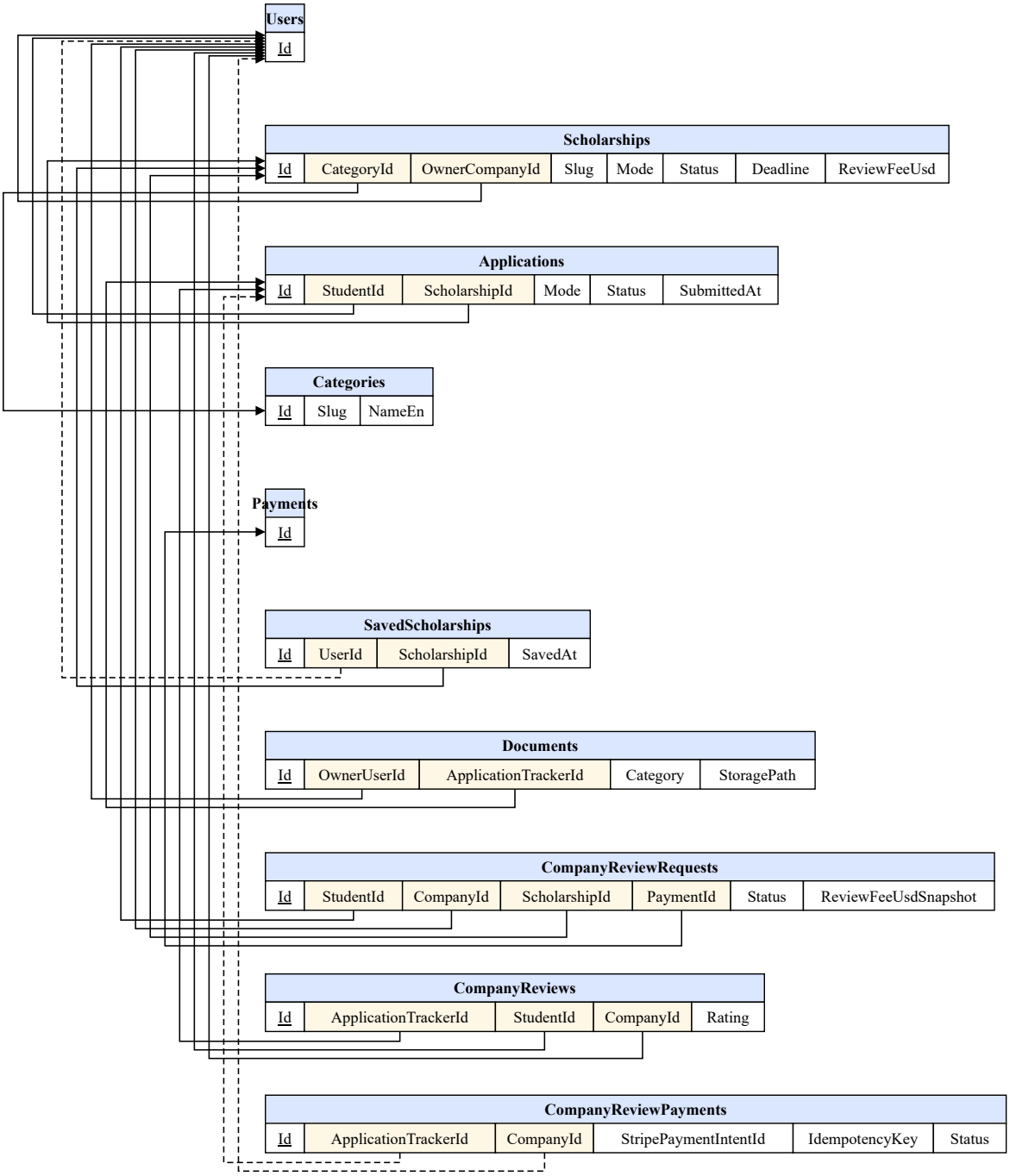

Figure 2: Relational mapping — Scholarships, Applications & Documents, showing the n-ary `CompanyReviewRequests` relation and the filtered-unique business rules.

4.3 Consultant Booking, Payments & Ratings

This area contains `Availabilities`→`Bookings` (1:N), the `Bookings`–`ConsultantReviews` 1:1 (Step 3), the financial-settlement relations (`Payments`, `Payouts`), and the session-recording trail. It is the strongest illustration of two policies: *all* FKs into `Users` are **Restrict** (a booking references `Users` twice — student and consultant), and money for settlement is stored as integer cents.

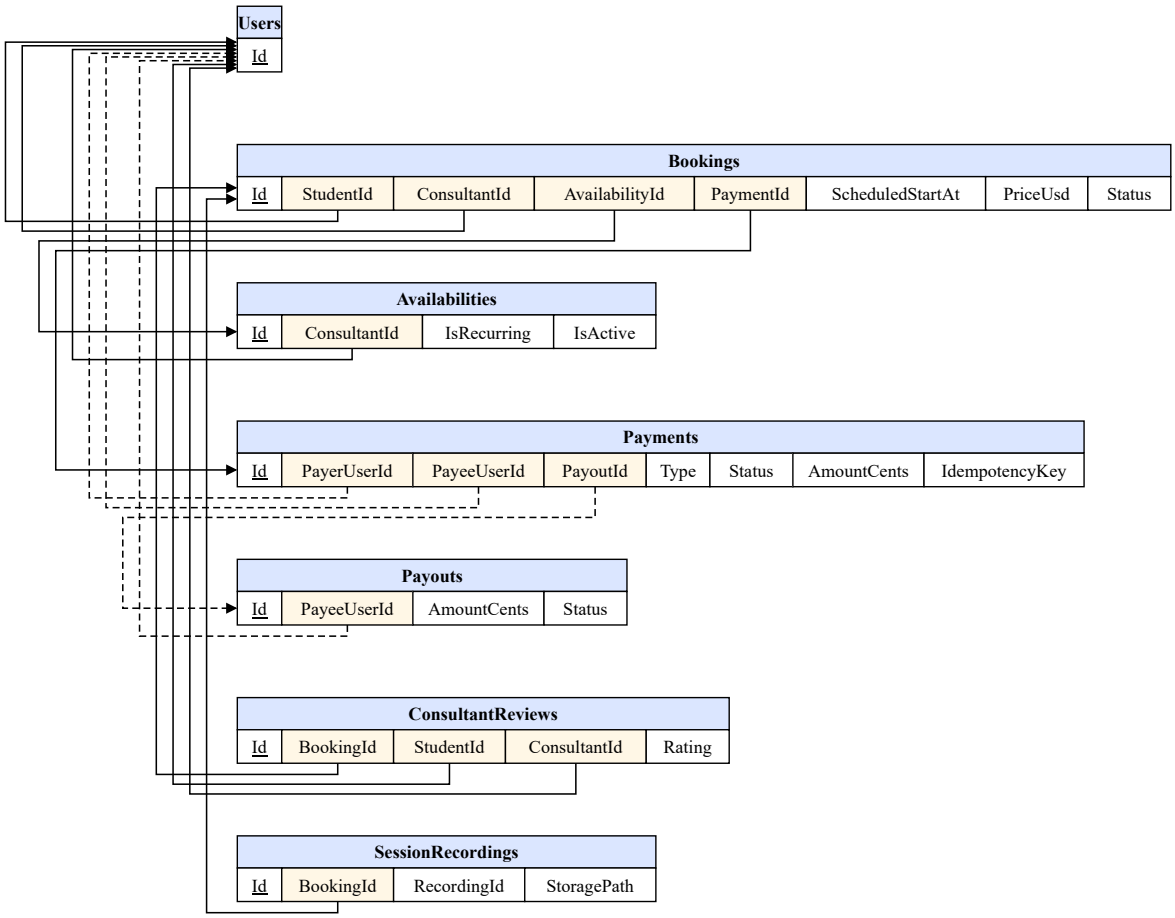

Figure 3: Relational mapping — Booking, Payments & Ratings. `Payments` and `Payouts` carry *no* database foreign keys by design (loose references); the booking's `(ConsultantId, ScheduledStartAt)` filtered-unique index prevents double-booking.

4.4 Community & Chat

The forum demonstrates a self-referencing 1:N (`ForumPosts` reply tree), the `ForumPostTags` M:N junction (Step 5), and several cascading child relations (votes, flags, bookmarks, attachments). The chat side (`Conversations`, `Messages`, `UserBlocks`) leans heavily on loose references to keep messaging decoupled from the user principal.

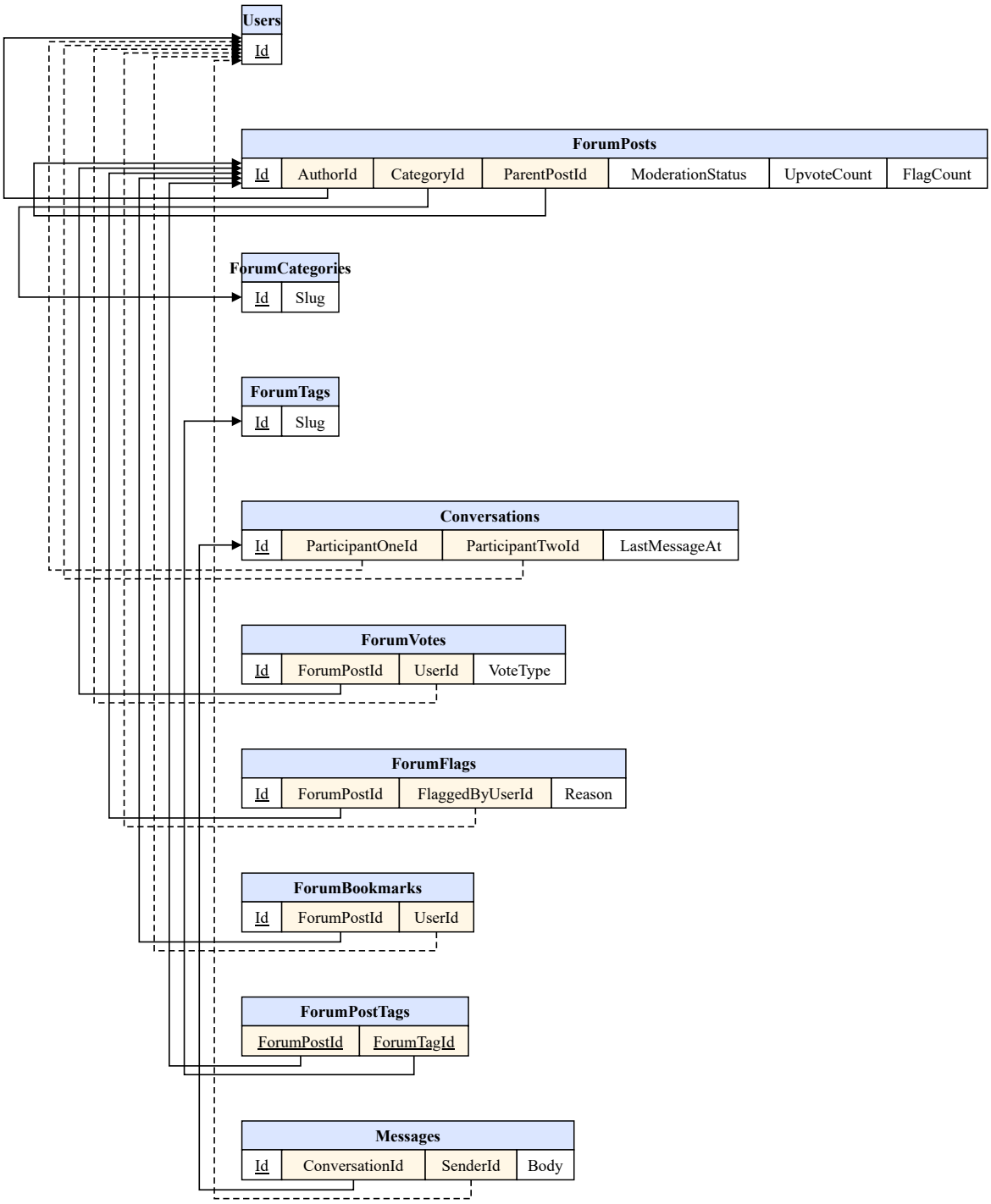

Figure 4: Relational mapping — Community & Chat. `ForumPostTags` is the true M:N junction; the *user* leg of votes/flags/bookmarks and the chat participant/sender legs are loose references.

4.5 Resources Hub & Notifications

`Resources`→`ResourceChapters` is a cascading 1:N with `ResourceChapters` as a weak entity (Step 2); progress and bookmark tables hang off `Resources` with cascade, while their `UserId` legs are loose. Notifications and notification preferences are fully loose-referenced.

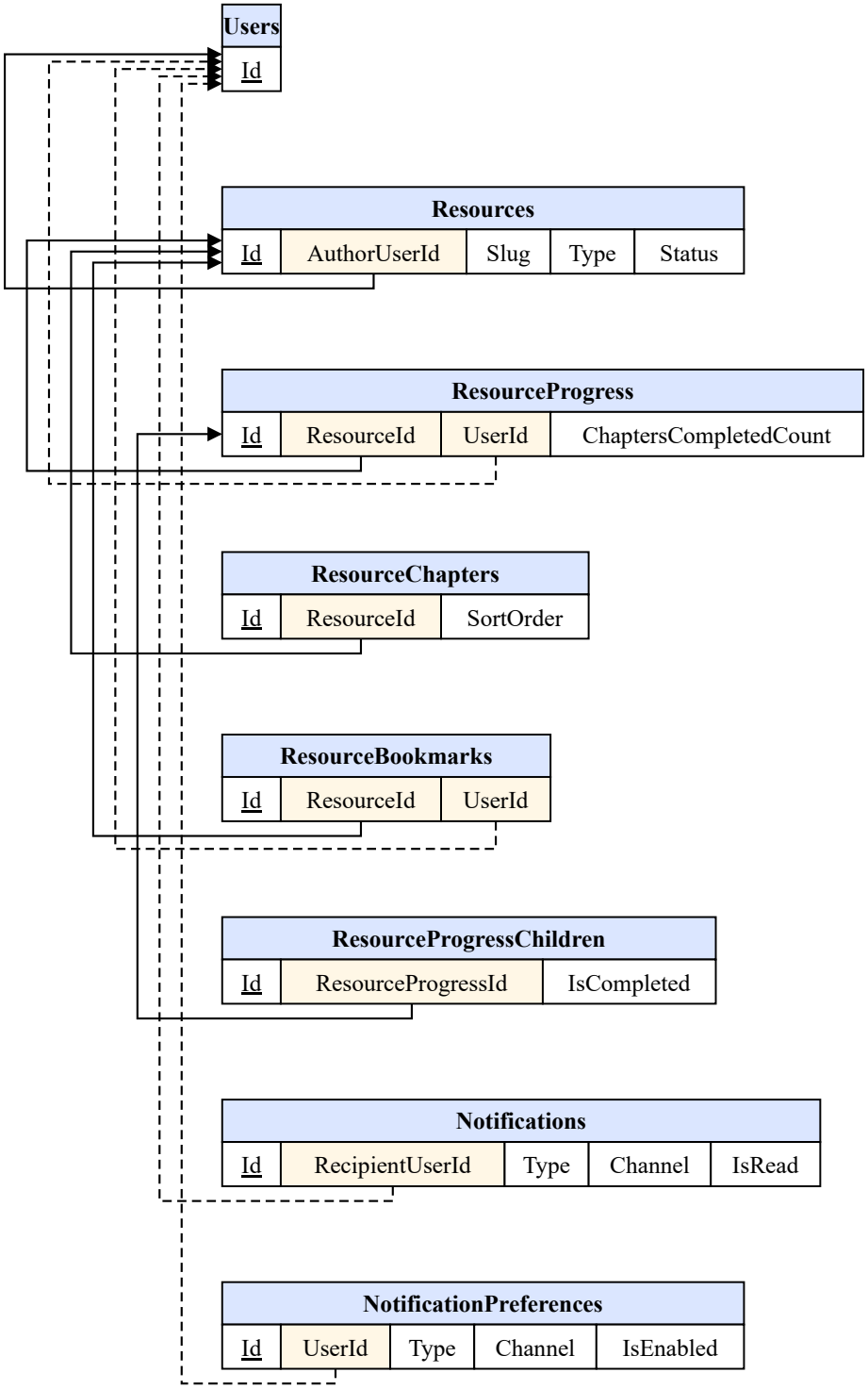

Figure 5: Relational mapping — Resources Hub & Notifications, showing the cascading chapter/progress children and the loose-referenced notification tables.

4.6 AI, Knowledge Base & Platform

The cross-cutting area holds `AiInteractions` with its 1:1 `AiRedactionAuditSamples` (Step 3), the `RecommendationClickEvents` fact table, the `KnowledgeDocuments` vector store, and the platform tables (`PlatformSettings`, `AuditLogs`, `UserDataRequests`, `UserRiskFlags`). Most of these use

loose references because they are high-volume, audit, or analytics relations.

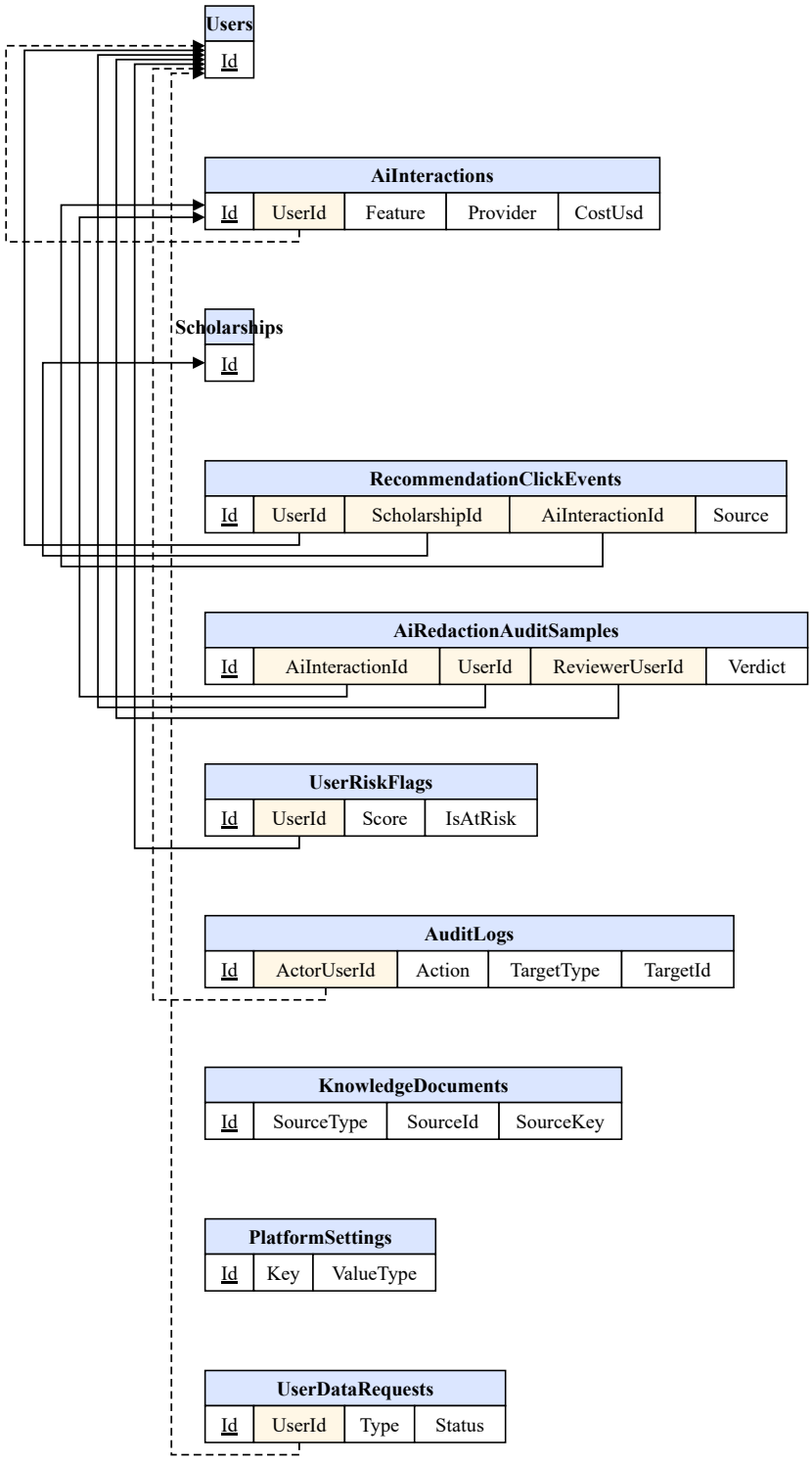

Figure 6: Relational mapping — AI, Knowledge Base & Platform. `AuditLogs` and `KnowledgeDocuments` use polymorphic soft references; `UserRiskFlags` is a 1:1 analytics-fed relation.

5 Referential-integrity strategy

Foreign-key delete rules are not an afterthought; they encode how the data model behaves under deletion and anonymization. ScholarPath uses four strategies, summarized in Table 3.

Table 3: Delete-rule strategy across the schema.

Rule	Used for	Why
Cascade	Education(→profile); upgrade files/links(→request); scholarship-children & saved-scholarships(→scholarship); application-children(→application); forum attachments/votes/flags/bookmarks/post-tags(→post/tag); messages(→conversation); resource chapters/bookmarks/progress/progress-children(→resource).	The child has no meaning without its parent (existence dependency).
Restrict	*Every* FK into `Users` (profile, refresh/reset tokens, upgrade-request, user-risk-flag, applications, documents, bookings, all reviews, recordings, recommendation clicks, redaction samples).	Protect history; and avoid SQL Server's *multiple cascade paths* error (1785) — a row may reference `Users` two or three times.
SetNull	scholarship→category, scholarship→owner-company, document→application, booking→availability/payment, request→payment, post→category.	Keep the row but orphan an *optional* link.
none (loose)	payments, payouts, notifications, the *user* leg of votes/flags/bookmarks, chat participants/sender, audit actor + target, AI `UserId`, knowledge `SourceId`, settings/requests/stories user.	Decouple high-volume / audit / analytics rows from the principal so deletes and anonymization never cascade-break.

The single most important defense point: **no foreign key cascades out of `Users`**. Every `..._Users_...` foreign key — including the 1:1 keys on `UserProfiles` and `UserRiskFlags` and the `UserRoles` join — is `ON DELETE NO ACTION` (Restrict). Cascading deletes originate only from the *aggregate roots* listed in the Cascade row (Scholarship, Application, ForumPost, Resource, Conversation, UpgradeRequest, and UserProfile→Education). This protects audit and financial history and side-steps SQL Server's multiple-cascade-paths restriction, since several relations reference `Users` more than once.

6 Accuracy: model versus live database

A graduation defense rewards precision about what the *deployed* database actually enforces, as opposed to what the conceptual model implies. Three points were verified directly against the live Azure SQL dump and deserve emphasis, because each is a place where the database does *not* match the naive reading of the model.

1. **The single-active-application rule (FR-057) is enforced in application code only.** There is *no* DB filtered-unique index on `Applications` in production. The historical `UX_Applications_Stude` index was dropped when `ScholarshipId` became nullable (migration `20260521163702`, to allow external-tracker applications with no scholarship) and was never recreated. So "one active application per (student, scholarship)" is a guarantee made by the service layer, not the schema.

2. **By contrast, `CompanyReviewRequests` and `Bookings` *do* have DB-enforced filtered-unique indexes.** `CompanyReviewRequests` keeps `(StudentId, ScholarshipId)` unique `WHERE Status IN ('Draft','Submitted','Pending','UnderReview')`, and `Bookings` keeps `(ConsultantId, ScheduledStartAt)` unique `WHERE Status IN ('Requested','Confirmed')`. These business rules are guaranteed by the database, not merely by code. The contrast with point (1) is the kind of detail a committee will probe, so it is stated plainly.

3. **Money is stored two different ways, on purpose.** Settlement-grade money (the `Payments` and `Payouts` relations: `AmountCents`, `PayeeAmountCents`, `RefundedAmountCents`, etc.) is stored as `bigint` *integer cents*, the standard way to avoid floating-point and rounding errors in financial settlement. Display-grade money (prices and fees: `PriceUsd`, `SessionFeeUsd`, `ReviewFeeUsd`, `FundingAmountUsd`) is stored as `decimal(p,2)`. The split is deliberate: cents for arithmetic that must reconcile to the last unit, decimal for catalog values that are only ever displayed or compared.

A handful of relations are also documented as *schema-only / planned* or *legacy / dead* (e.g. `ForumPostAttachments`, `SuccessStories`, and `CompanyReviewPayments`); they exist in the schema for completeness but have no live write path. Documenting them as such — rather than pretending they are active — is part of keeping the mapping honest.

If a reviewer wants the authoritative DDL, the project can regenerate it straight from the migrations:

```
cd server
dotnet ef migrations script \
  --project src/ScholarPath.Infrastructure \
  --startup-project src/ScholarPath.API \
  > ../docs/diagrams/schema.sql
```

7 Closing

Applying Elmasri & Navathe's seven/nine-step algorithm to the Part 2 EERD yields the ScholarPath relational schema: **62 tables deployed** to Azure SQL, every one traceable back to a specific conceptual construct via Table 1. Strong entities became relations, weak entities became cascading child relations, 1:1 links became unique foreign keys, M:N relationships became junction tables, the user hierarchy became a single discriminated table, and the n-ary review workflow became a four-foreign-key relation — with deliberate, documented deviations (JSON multivalued columns, loose references) where the deployed schema favors pragmatism over textbook purity.

This completes the relational view. The *object-oriented* view — how these relations map back to C# entity classes, aggregates, and navigation properties — is the subject of *ScholarPath — Class Diagrams* (Part 4).